Отчет по индивидуальному заданию №2

ФИО: Куделькина Дарья Дмитриевна
Группа: БПИ-253
Вариант: 44

Условие задачи

Предприятие выпускает изделия, состоящие из нескольких деталей. Для каждой детали задан технологический маршрут: последовательность операций на станках определённых типов. Готовые детали поступают в промежуточные накопители, после чего сборочный пост может начать работу только при наличии полного комплекта компонентов. После обработки деталь проходит контроль качества. Годная деталь передаётся дальше, бракованная направляется на повторную обработку или списывается. Вместимость накопителей ограничена, поэтому заполненный выходной буфер способен приостановить станок. Транспортные роботы перемещают детали между стадиями. Требуется разработать консольное приложение, моделирующее сборочную линию.

Входные параметры: перечень изделий и состав комплектов; технологические маршруты деталей; количество станков и сборочных постов; число транспортных роботов; длительность операций; вместимость накопителей; вероятность брака и правила повторной обработки; план выпуска изделий.

Отображаемые события: создание детали, постановка в очередь, начало и окончание операции, контроль качества, обнаружение брака, повторная обработка, перемещение, поступление в комплект, начало сборки и выпуск изделия.

Инварианты: станок обрабатывает не более одной детали; деталь находится на одной стадии или у одного транспортного робота; операция выполняется после всех предыдущих операций маршрута; забракованная деталь не входит в готовое изделие; сборка начинается только при наличии полного допустимого комплекта.

Завершение: после выполнения плана, исчерпания исходных деталей либо признания дальнейшего производства невозможным. Станки, контролёры, сборочные посты, транспортные роботы и детали являются самостоятельными участниками модели.

Анализ сущностей: что может быть потоком

Станок, контролёр ОТК, транспортный робот и сборочный пост реализованы как потоки. Каждый из них работает по одному циклу: дождаться работы, взять деталь, некоторое время её обрабатывать и передать дальше. Такой цикл естественно ложится на поток: ожидание - блокировка на синхропримитиве, обработка - сон потока на время операции.

Склад заготовок через равные промежутки времени выпускает новые заготовки, а когда выпускать нечего - ждёт, не спишут ли деталь, которую нужно заменить. Поэтому

склад тоже реализован как отдельный поток.

Деталь ничего не делает сама, её перемещают и обрабатывают другие. Если сделать каждую деталь потоком, такой поток всё время только ждал бы, пока его возьмут, а реальные действия всё равно выполнялись бы потоками станков и роботов. Число деталей к тому же заранее неизвестно и может достигать сотен. Поэтому деталь реализована как данные - структура с номером, типом, позицией в маршруте и местом нахождения, - которыми владеет тот поток, у которого деталь сейчас находится.

Накопители (очереди к станкам, выходные накопители, вход и выход ОТК, комплекты) поведения не имеют. Это общие ресурсы с ограниченной вместимостью, через которые потоки передают друг другу детали. Они реализованы как разделяемые структуры.

Изделие - результат работы сборочного поста, поведения не имеет и хранится как счётчик выпуска.

Помимо участников модели, в программе есть два служебных потока. Поток сигналов принимает сигналы операционной системы и меняет условия завершения. Главный поток после запуска остальных работает «сторожем»: следит за лимитом времени и замечает, что линия встала.

Итого при базовой конфигурации работает 10 потоков участников (склад, 5 станков, 1 контролёр, 2 робота, 1 пост), поток сигналов и главный поток.

Основные сущности программы

Сборочная линия (Line)

Вся линия хранится в одной глобальной структуре Line, которую видят все потоки: детали, накопители, станки, контролёры, роботы, посты, счётчики выпуска, флаги остановки и причина завершения. Структуру защищает один мьютекс line.lock. Любое чтение или изменение общих данных выполняется только при захваченном мьютексе.

Время в модели измеряется в условных единицах: одна единица модельного времени равна заданному числу миллисекунд реального времени (по умолчанию 50, ключ -d). Операция длительностью 4 единицы означает, что поток станка спит 4 * 50 = 200 мс. Модельное время события вычисляется как прошедшее реальное время, делённое на длительность единицы.

По умолчанию моделируется участок выпуска редукторов: редуктор (REDUCER) собирается из вала (SHAFT), двух шестерён (GEAR) и корпуса (HOUSING). Детали обрабатываются на токарном (LATHE), двух фрезерных (MILL), сверлильном (DRILL) и шлифовальном (GRIND) станках.

Деталь (Part)

Деталь - данные, а не поток. У детали есть тип, номер следующей операции маршрута, число переделок, отметка о прохождении контроля и место, где она

находится. Деталь в каждый момент находится ровно в одном месте: на складе, в очереди к станку, на станке, в выходном накопителе станка, у робота, на входе ОТК, у контролёра, на выходе ОТК, в комплекте, на сборке, в готовом изделии или в списании. Перемещение детали из одного места в другое всегда выполняется под мьютексом.

Накопитель (Store)

Накопитель - ограниченный буфер деталей. Кроме массива деталей, у каждого накопителя есть два счётчика ожидания. Счётчик free равен числу свободных мест: тот, кто хочет положить деталь, сначала уменьшает его, а тот, кто забрал деталь, увеличивает. Счётчик ready равен числу деталей, которые можно взять: его увеличивает робот после доставки, а уменьшает станок или контролёр, ожидающий работу. У склада заготовок вместимость не ограничена.

Склад заготовок (поток warehouse_thread)

Раз в release_interval единиц склад выбирает, деталь какого типа нужна (по очереди между типами, только если заготовки этого типа ещё есть и деталей на линии вместе с уже собранными меньше, чем требует план), создаёт её и сообщает роботам, что появилась работа. Если ничего выпускать не нужно, склад засыпает на счётчике release_wake и просыпается, когда контролёр списывает деталь и её нужно заменить.

Станок (поток machine_thread)

Станок ждёт на счётчике ready очереди своего типа, забирает первую деталь и проверяет, что следующая операция её маршрута выполняется именно на станке этого типа. Затем он освобождает место в очереди, отпускает мьютекс и спит на время операции. После операции увеличивает номер шага маршрута и пытается занять место в своём выходном накопителе. Если накопитель полон, станок сообщает о приостановке и ждёт, пока робот не заберёт деталь. Время простоя учитывается в статистике.

Контролёр ОТК (поток inspector_thread)

Контролёр ждёт деталь на входе ОТК, проверяет её и случайным образом решает, есть ли брак, с вероятностью, заданной для типа детали. Для случайных чисел используется функция rand_r с собственным состоянием у каждого контролёра. Годная деталь получает отметку и попадает на выход ОТК. Бракованная либо возвращается на заданную операцию маршрута (если переделка разрешена и лимит не исчерпан), либо списывается, и тогда контролёр будит склад, чтобы тот выпустил замену. Если выход ОТК полон, контролёр ждёт места так же, как станок.

Транспортный робот (поток robot_thread)

Робот спит на общем счётчике robot_work, который увеличивает любой поток, когда у роботов могла появиться работа. Проснувшись, робот под мьютексом ищет деталь, которую можно везти: сначала на выходе ОТК, затем в выходных накопителях

станков, затем на складе - чтобы сначала разгружать линию, а потом подавать на неё новое. Деталь везут только туда, где для неё есть место: место назначения бронируется заранее неблокирующим уменьшением счётчика free. Если места нет, робот смотрит следующую деталь. Затем робот забирает деталь, отпускает мьютекс, спит время перевозки и кладёт деталь в забронированное место. После доставки он будит потребителя: станок или ОТК - через счётчик ready, сборочные посты - через счётчик kit_event.

Сборочный пост (поток station_thread)

Пост спит на счётчике kit_event. Проснувшись, он под мьютексом проверяет, есть ли полный комплект для какого-нибудь изделия, план по которому ещё не выполнен, и забирает весь комплект сразу, одной операцией под мьютексом. Поэтому два поста не могут разобрать один комплект пополам. После сборки пост увеличивает счётчик выпуска, и если план выполнен по всем изделиям, останавливает линию.

Поток сигналов и сторож

Поток сигналов в бесконечном цикле вызывает sigwait и обрабатывает полученный сигнал обычным кодом, без ограничений, которые накладываются на асинхронные обработчики. Главный поток после запуска всех участников раз в единицу модельного времени проверяет лимит времени и время последнего события. Если событий нет дольше, чем (2 * самая длинная операция + 5) единиц, линия встала: все потоки ждут друг друга. Тогда сторож определяет причину (исчерпание заготовок или взаимная блокировка) и останавливает программу.

Мьютекс

Все общие данные защищает один мьютекс line.lock. Один мьютекс вместо отдельного для каждого накопителя выбран сознательно: робот при выборе работы смотрит сразу в несколько накопителей (источник и место назначения), а пост - во все комплекты. С несколькими мьютексами пришлось бы захватывать их в строго определённом порядке, чтобы не получить взаимную блокировку, а с одним такой проблемы нет. Критические секции короткие (переложить деталь, изменить счётчики, напечатать событие), а всё долгое - операции, перевозка, сборка - выполняется без мьютекса, поэтому потоки работают параллельно и не мешают друг другу.

В программе соблюдаются два правила: ждать на семафоре можно только без мьютекса, и спать во время работы можно только без мьютекса. Нарушение первого правила привело бы к тому, что поток уснул бы, не дав никому другому изменить данные, которых он ждёт.

Семафоры

Ожидание организовано на семафорах, в программе они называются счётчиками (Counter). Используются два вида счётчиков.

Счётчики ресурсов - free и ready у каждого накопителя. Производитель ждёт свободное место, потребитель ждёт готовую деталь. Станок блокируется на ready

своей очереди, если деталей нет, и на free выходного накопителя, если тот заполнен. Робот, наоборот, никогда не блокируется на free, а делает неблокирующую попытку (sem_trywait): получилось - место забронировано, нет - ищем другую деталь.

Счётчики событий - robot_work, kit_event и release_wake. Они значат «что-то изменилось, надо проверить ещё раз». Проснувшийся поток под мьютексом сам проверяет, есть ли для него работа, и если нет, снова засыпает. Лишнее пробуждение безопасно, а пропустить нужное невозможно, потому что увеличение счётчика запоминается, даже если в этот момент никто не ждёт.

Пример передачи детали от станка LATHE-1 к станку GRIND-1 (wait - ожидание на семафоре, trywait - попытка без ожидания, post - увеличение семафора, lock и unlock - захват и освобождение мьютекса):

1. Станок GRIND-1 ждёт работу: wait(очередь GRIND.ready).
2. Станок LATHE-1 выполняет операцию без мьютекса, затем trywait(выход.free), lock, кладёт деталь в выходной накопитель, unlock и post(robot_work).
3. Робот просыпается в wait(robot_work), делает lock, находит деталь и бронирует место: trywait(очередь GRIND.free).
4. Робот забирает деталь, делает unlock и post(выход.free) - у LATHE-1 освободилось место.
5. Робот везёт деталь без мьютекса, затем lock, кладёт её в очередь GRIND, unlock и post(очередь GRIND.ready).
6. Станок GRIND-1 просыпается, под мьютексом берёт деталь и начинает операцию.

Остановка потоков

Когда нужно завершить работу (план выполнен, получен сигнал, сторож обнаружил остановку линии), вызывается функция stop_all. Она под мьютексом ставит флаг stop, записывает причину (только первую - если причин несколько, сохраняется та, что произошла раньше) и увеличивает все счётчики, на которых может спать какой-нибудь поток. Каждый поток, проснувшись, первым делом проверяет флаг stop и выходит из цикла. Главный поток дожидается всех участников вызовом pthread_join, после чего завершает поток сигналов, проверяет правила и выводит итоги.

Сигналы

Перед созданием потоков главный поток блокирует сигналы SIGINT, SIGTERM, SIGUSR1 и SIGUSR2 функцией pthread_sigmask. Созданные потоки наследуют эту маску, поэтому сигнал не прервёт случайный поток посреди работы с данными. Все эти сигналы принимает только поток сигналов через sigwait. SIGINT и SIGTERM останавливают линию. SIGUSR1 печатает текущее состояние линии, и работа продолжается. SIGUSR2 меняет условие завершения: склад перестаёт выпускать новые заготовки, а детали, которые уже на линии, доделываются. Когда всё, что можно было собрать из этих деталей, собрано и линия затихла, программа завершается.

Вторая программа: условные переменные

Сценарий взаимодействия написан через небольшой интерфейс «счётчик» (файл counter.h): инициализировать, ждать (уменьшить, а если значение ноль - заснуть), попробовать уменьшить без ожидания, увеличить. У интерфейса две реализации, и из одного исходного кода собираются две программы.

Программа line_sem (файл counter_sem.c) использует семафоры POSIX: sem_init, sem_wait, sem_trywait, sem_post.

Программа line_cond (файл counter_cond.c) использует мьютекс и условную переменную. Счётчик хранит целое значение под собственным мьютексом. Ожидание - цикл while (value == 0) pthread_cond_wait, затем уменьшение; увеличение - инкремент и pthread_cond_signal. Цикл while, а не if, нужен из-за возможных ложных пробуждений.

Обе программы ведут себя одинаково и проходят одни и те же тесты.

Правила, которые соблюдаются всегда

Станок обрабатывает не более одной детали; деталь находится на одной стадии или у одного робота; операция выполняется только после всех предыдущих операций маршрута; забракованная деталь не входит в готовое изделие; сборка начинается только при наличии полного комплекта.

В многопоточной программе эти правила обеспечиваются тем, что любое перемещение детали - это одно действие под мьютексом, а вместимость накопителей не может быть превышена, потому что положить деталь можно, только получив место через счётчик free. Кроме того, правила проверяются. Во время работы станок проверяет, что пришедшая к нему деталь должна выполнять операцию именно на станке этого типа и что он сейчас свободен; пост проверяет, что каждая деталь комплекта прошла контроль. После завершения всех потоков выполняется полная проверка: обходятся все накопители, станки, контролёры, роботы и посты, и для каждой детали считается, в скольких местах она найдена. Деталь на линии должна встретиться ровно один раз, вошедшая в изделие или списанная - ни разу. Также проверяется, что ни один накопитель не переполнен и что место, записанное в детали, совпадает с тем, где она лежит. О любом нарушении программа сообщает событием «!ИНВАРИАНТ».

Входные данные

Без указания файла используется встроенная конфигурация - участок выпуска редукторов. Файл конфигурации читается системными вызовами open, read и close, символ # начинает комментарий:

```
machine LATHE 1
part SHAFT raw 6 defect 0.10 rework 2 max_rework 1 route LATHE:4 GRIND:3
product REDUCER plan 4 kit SHAFT:1 GEAR:2 HOUSING:1
robots 2
```

Строка machine задаёт тип станка и количество станков данного типа - от него зависит число потоков-станков. Строка part описывает тип детали: запас заготовок (raw), вероятность брака (defect), с какой операции переделывать брак (rework, или none - сразу списывать), сколько раз можно переделывать (max_rework) и маршрут (route) в виде «станок:длительность». Строка product задаёт изделие: план выпуска (plan) и состав комплекта (kit). Остальные параметры: число роботов и время перевозки (robots, transport_time), число контролёров и время проверки (inspectors, inspect_time), число постов и время сборки (stations, assembly_time), вместимости накопителей, интервал выпуска заготовок (release_interval), начальное значение генератора случайных чисел (seed) и лимит модельного времени (max_time). Число потоков каждого вида задаётся этими же параметрами.

При синтаксической ошибке программа сообщает имя файла, номер строки и причину. Затем проверяется смысл данных, и если изделие собрать заведомо невозможно, программа сообщает об этом, не запуская потоки.

Пользовательский интерфейс

Запуск

cd HW-2
make
./bin/line_sem [-c файл] [-s seed] [-d мс] [-l журнал] [-q] [-t лимит]
./bin/line_cond [те же ключи]

Ключ -c задаёт файл конфигурации. Ключ -s задаёт начальное значение генератора случайных чисел. Ключ -d задаёт, сколько миллисекунд длится единица модельного времени: чем больше значение, тем медленнее идёт моделирование и тем удобнее следить за событиями. Ключ -l задаёт файл журнала (по умолчанию line.log, «-l -» - без журнала). Ключ -q включает тихий режим, когда на экран выводятся только итоги. Ключ -t ограничивает модельное время. Скрипт tests/demo.sh запускает базовый сценарий медленно (150 мс на единицу) и печатает PID процесса, чтобы можно было посылать ему сигналы.

Управление во время работы - сигналами из другого терминала:

kill -USR1 <pid> показать состояние линии
kill -USR2 <pid> мягкая остановка
kill <pid> остановка (SIGTERM), то же, что Ctrl+C

Что выводит программа

В начале выводится сводка параметров, затем строка запуска с числом потоков:

[t= 0] СИСТЕМА Линия запуск: станков 5, контролёров 1, роботов 2, постов 1 - 10 потоков

Во время работы каждое событие выводится отдельной строкой: модельное время, вид события, поток-участник и описание. Виды событий: СОЗДАНИЕ, ОЧЕРЕДЬ, ПЕРЕВОЗКА, ОП.НАЧАЛО, ОП.КОНЕЦ, КОНТРОЛЬ, БРАК, ПЕРЕДЕЛКА,

СПИСАНИЕ, КОМПЛЕКТ, СБОРКА, ВЫПУСК, ПРОСТОЙ (станок или контролёр ждёт места) и СИСТЕМА (запуск, получен сигнал). Так как события печатают разные потоки, вывод защищён отдельным мьютексом, и строки разных потоков никогда не перемешиваются. Вывод выполняется через файловые дескрипторы системным вызовом write.

По сигналу SIGUSR1 печатается снимок состояния линии:

```
--- Состояние линии, t=40 ---
Выпуск: REDUCER 0/4
Склад: 3, вход ОТК: 0/3, выход ОТК: 0/2
Станки: LATHE-1[SHAFT#10] MILL-1[GEAR#11] MILL-2[HOUSING#9] DRILL-
    1[GEAR#8] GRIND-1[-]
Роботы: РОБОТ-1[HOUSING#12] РОБОТ-2[GEAR#13]
Комплекты: SHAFT 1/4 GEAR 0/4 HOUSING 1/4
---
```

В конце при любом способе завершения выводятся итоги: причина завершения; время работы в модельных единицах и секундах и число потоков; число событий и использованное начальное значение генератора; число проверок и нарушений правил; выпуск по каждому изделию, средний цикл изготовления изделия и такт выпуска; по каждому типу детали - сколько создано, забраковано, переделано, списано, вошло в изделия и средний путь от склада до комплекта; по каждому потоку участнику - число операций и доля времени работы и простоя; наибольшее заполнение склада и накопителей ОТК. По долям времени видно узкое место линии. Параметры, события и итоги одновременно записываются в журнал.

Варианты завершения программы

Каждому варианту соответствует свой код завершения.

План выполнен - все изделия по плану выпущены. Последний сборочный пост останавливает линию. Код 0.

Ошибка в данных - неверный ключ, файл не найден или в нём ошибка. Потоки не запускаются. Код 1.

Исходные детали исчерпаны. Линия затихла, а для какого-то типа детали заготовок не осталось, и до плана не хватает. Это определяет сторож. Код 2.

Дальнейшее производство невозможно. Невозможность может быть видна по исходным данным (нет нужного станка, комплект не помещается в накопитель и т. п.) - тогда потоки не запускаются. Либо линия может встать в процессе работы: детали заполнили накопители так, что все потоки ждут друг друга (взаимная блокировка модели). Сторож замечает, что событий давно нет, а детали остались на линии, и перечисляет заблокированных участников. Важно, что это взаимная блокировка участников модели, а не потоков программы: потоки при этом не зависают, они штатно ждут на семафорах и завершаются по команде сторожа. Код 3.

Превышен лимит модельного времени (ключ -t). Код 4.

Нарушены правила модели, возможно только при ошибке в программе. Код 5.

Прерывание сигналом SIGINT (Ctrl+C) или SIGTERM. Линия останавливается, потоки завершаются, итоги выводятся и записываются в журнал. Коды 130 и 143.

Мягкая остановка по сигналу SIGUSR2. Склад перестаёт выпускать заготовки, детали в работе доделываются, после чего программа завершается. Код 140. Если оставшихся на линии деталей хватило на выполнение плана, итог - «план выполнен», код 0.

Структура программы

Файлы config.h и config.c - параметры, встроенная конфигурация, чтение и проверка файла (без изменений по сравнению с заданием № 1). Файл model.h - структуры участников и линии. Файл line.c - функции потоков склада, станка, контролёра, робота и поста, остановка, диагностика и проверка правил. Файл main.c - разбор ключей, журнал, поток сигналов, создание и ожидание потоков, сторож, итоги и код завершения. Файл counter.h - интерфейс счётчика, counter_sem.c и counter_cond.c - две его реализации. Файлы log.h и log.c - потокобезопасный вывод событий.

Программа компилируется командой make без предупреждений с ключами -Wall -Wextra -Wpedantic. Все файлы, структуры и функции снабжены комментариями.

Результаты работы

Базовый сценарий

Прогон bin/line_sem -c configs/01_basic.cfg -d 20. Фрагмент вывода, в котором видны брак, списание, переделка и приостановка станка:

```
[t= 47] ПРОСТОЙ GRIND-1 выходной накопитель заполнен - станок
    приостановлен, GEAR#8 на станке
[t= 51] ПРОСТОЙ GRIND-1 место освободилось - станок возобновляет работу
[t= 52] БРАК ОТК-1 в HOUSING#9 обнаружен брак
[t= 52] СПИСАНИЕ ОТК-1 HOUSING#9 списана: переделка не предусмотрена
[t= 54] БРАК ОТК-1 в SHAFT#10 обнаружен брак
[t= 54] ПЕРЕДЕЛКА ОТК-1 SHAFT#10 возвращается на операцию 2 (GRIND),
    переделка 1 из 1
...
[t= 86] СБОРКА ПОСТ-1 начало сборки REDUCER №4 из комплекта: SHAFT#10,
    GEAR#15, GEAR#16, HOUSING#17
[t= 91] ВЫПУСК ПОСТ-1 выпущено изделие REDUCER №4, по плану 4/4
```

Итоги:

```
Завершение: ПЛАН ВЫПОЛНЕН
Причина: все изделия по плану выпущены
Время работы: 93 ед. (1.9 с), потоков участников: 10
Событий: 449, seed: 13
Проверок правил: 134, нарушений правил модели: 0
```

Изделие REDUCER: выпущено 4 из 4
Средний цикл изделия: 58.2, такт выпуска: 23.2
Деталь SHAFT: создано 4, брак 1, переделок 1, списано 0, в изделиях 4, на линии 0, ср. путь до комплекта 29.5
Деталь GEAR: создано 8, брак 0, переделок 0, списано 0, в изделиях 8, на линии 0, ср. путь до комплекта 42.0
Деталь HOUSING: создано 5, брак 1, переделок 0, списано 1, в изделиях 4, на линии 0, ср. путь до комплекта 26.8
Станок LATHE-1: операций 4, работа 17.1%, приостановлен 0.0%
Станок MILL-1: операций 7, работа 38.5%, приостановлен 0.0%
Станок MILL-2: операций 6, работа 36.4%, приостановлен 0.0%
Станок DRILL-1: операций 13, работа 33.2%, приостановлен 1.1%
Станок GRIND-1: операций 13, работа 33.2%, приостановлен 9.7%
Контролёр ОТК-1: проверено 18, брак 2, работа 19.3%, ожидание места 0.0%
Робот РОБОТ-1: рейсов 39, работа 83.4%
Робот РОБОТ-2: рейсов 38, работа 81.3%
Пост ПОСТ-1: собрано 4, работа 21.4%
Максимальное заполнение: склад 8, вход ОТК 1/3, выход ОТК 1/2

Вал SHAFT#10 был забракован, вернулся на шлифовку и затем вошёл в четвёртый редуктор. Списанный корпус HOUSING#9 заменён новой заготовкой (создано 5 корпусов при плане 4). Узкое место линии - транспорт: роботы заняты более 80 % времени, станки - 17–39 %. Шлифовальный станок GRIND-1 почти 10 % времени простоял с готовой деталью, потому что роботы не успевали её забрать. Изделие в среднем изготавливается за 58 единиц от создания первой детали комплекта до выпуска, а новый редуктор выходит в среднем раз в 23 единицы. Видно и настоящее параллельное выполнение: суммарная занятость всех участников - около 370 %, то есть потоки действительно работали одновременно.

Программа line_cond на том же сценарии выдаёт аналогичный результат (план выполнен за 102 единицы, роботы заняты 76–80 %). Точные числа от запуска к запуску немного различаются: потоки планируются операционной системой, поэтому порядок, в котором детали попадают к контролёру, а значит, и то, какой детали достанется брак, каждый раз может быть другим. Это ожидаемое свойство многопоточной модели; итог - выполнение плана - при этом стабилен.

Сигналы

Мягкая остановка: SIGUSR2 послан примерно на 23-й единице времени.

[t= 23] СИСТЕМА Линия SIGUSR2: новые заготовки не выпускаются, доделываем детали в работе
...
Завершение: МЯГКАЯ ОСТАНОВКА (SIGUSR2)
Причина: детали, которые были в работе, доделаны
Изделие REDUCER: выпущено 2 из 4

Код завершения 140. Из деталей, которые уже были на линии, собрано два редуктора.

Прерывание Ctrl+C:

Завершение: ПРЕРВАНО СИГНАЛОМ
Причина: получен сигнал SIGINT (Ctrl+C)
Время работы: 26 ед. (0.8 с), потоков участников: 10
Событий: 131, seed: 13
Проверок правил: 55, нарушений правил модели: 0

Код завершения 130. Все потоки завершились, итоги записаны в журнал.

Исчерпание заготовок

Сценарий configs/04_raw_exhausted.cfg - заготовок PLATE меньше, чем нужно по плану:

Завершение: ИСХОДНЫЕ ДЕТАЛИ ИСЧЕРПАНЫ
Причина: заготовки PLATE исчерпаны (списано 3)

Взаимная блокировка

Сценарий configs/05_deadlock.cfg. Быстрые детали LEFT заполняют комплект, выход и вход ОТК, и медленная деталь RIGHT, без которой комплект не собрать, уже не может попасть на контроль. Сторож замечает, что линия встала:

Завершение: ПРОИЗВОДСТВО НЕВОЗМОЖНО (взаимная блокировка)
Причина: на линии застряло деталей: 9; WELD-1 приостановлен; ОТК-1 ждёт места;

Тестирование

Используются те же десять тестовых наборов данных, что и в задании №1 (папка configs): работа с браком, переделкой и списанием; работа без брака; высокий брак с многократной переделкой; исчерпание заготовок; взаимная блокировка; отсутствие нужного станка; комплект, не помещающийся в накопитель; ошибка в файле; узкое место с приостановкой станков; два вида изделий с общей деталью и двумя постами.

Скрипт tests/run_tests.sh запускается командой make test и проверяет обе программы. Для каждого прогона проверяются код завершения, итоговое сообщение и отсутствие нарушений правил. Кроме того, скрипт проверяет:

- встроенную конфигурацию, лимит времени, несуществующий файл и неверный ключ;
- что в журнале базового сценария есть все обязательные виды событий;
- что при нулевой вероятности брака брака нет, при высокой - есть переделки, а в сценарии с узким местом станок приостанавливается;
- что базовый сценарий пять раз подряд выполняет план (так как результат многопоточной программы не повторяется в точности, проверяется устойчивость итога, а не совпадение журналов);
- реакцию на сигналы: SIGINT и SIGTERM (коды 130 и 143, итоги в журнале), SIGUSR1 (выводится состояние), SIGUSR2 (мягкая остановка, код 140).

Все 23 проверки проходят для каждой из двух программ. Дополнительно программы многократно запускались с очень коротким тактом (-d 3), когда потоки особенно активно конкурируют за мьютекс, - зависаний и нарушений правил не было. ThreadSanitizer гонок данных не обнаружил.